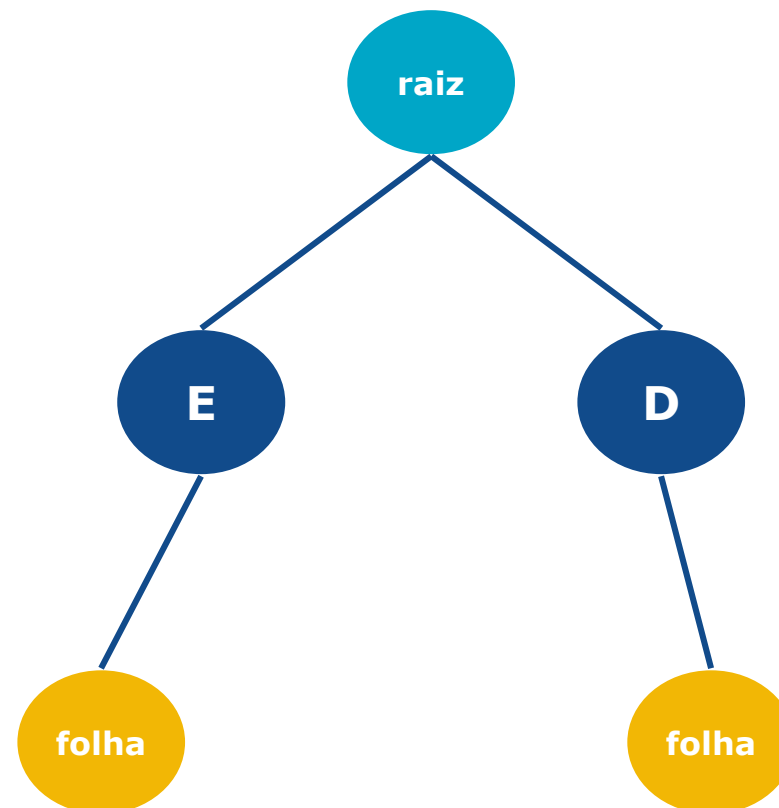


Árvores simples e busca básica

Estrutura de Dados II

Ciência da Computação • 14 de setembro • 2 horas

Prof.^a Kadidja Valéria



Roteiro da aula

Organização das atividades em 120 minutos

1. Conceitos

25 min

Vocabulário,
estrutura e
aplicações

2. ABB e busca

30 min

Regra de
ordenação,
inserção e busca

3. Python

20 min

Implementação
guiada e teste

4. Workshop

30 min

Desafio prático
em grupos

5. Pitches

15 min

Apresentação,
feedback e
síntese

Resultado esperado

Cada grupo entrega uma solução de busca representada por árvore, com regras, exemplo e justificativa.

Objetivos de aprendizagem

Ao final da aula, o estudante deverá ser capaz de

Compreender

Reconhecer raiz, nós, arestas, níveis, altura, folhas e subárvores em uma representação hierárquica.

Aplicar

Construir uma árvore binária de busca e executar uma busca passo a passo, justificando cada decisão.

Implementar

Representar nós em Python e testar uma função básica de busca em uma ABB.

Pergunta de abertura

Como organizar dados para evitar uma busca item por item?

Imagine um catálogo com 1.000 códigos.

Que decisões permitem descartar parte dos dados a cada comparação?

Tempo de discussão: 5 minutos

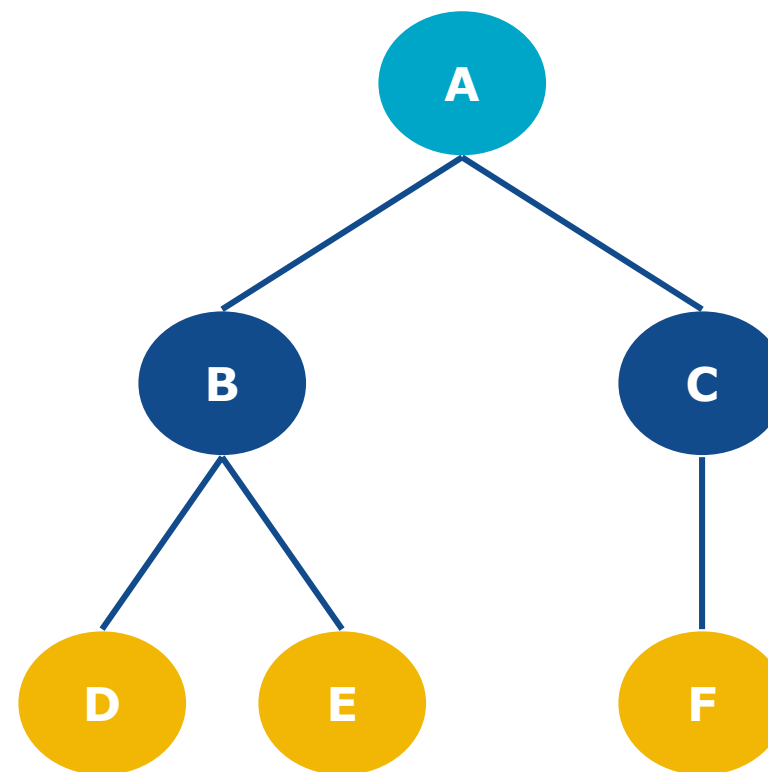
Converse em dupla

1. Cite uma organização hierárquica do cotidiano.
2. Explique como você procuraria um item nela.
3. Compartilhe uma estratégia com a turma.

Estrutura de dados em árvore

Uma organização não linear e hierárquica

- Cada elemento ocupa um nó.
- As ligações entre nós formam relações de parentesco.
- A estrutura começa em uma raiz e se divide em subárvores.
- Não há ciclos em uma árvore simples.



A hierarquia define os caminhos possíveis entre os elementos.

Vocabulário essencial

Os termos descrevem posição e relacionamento

Raiz

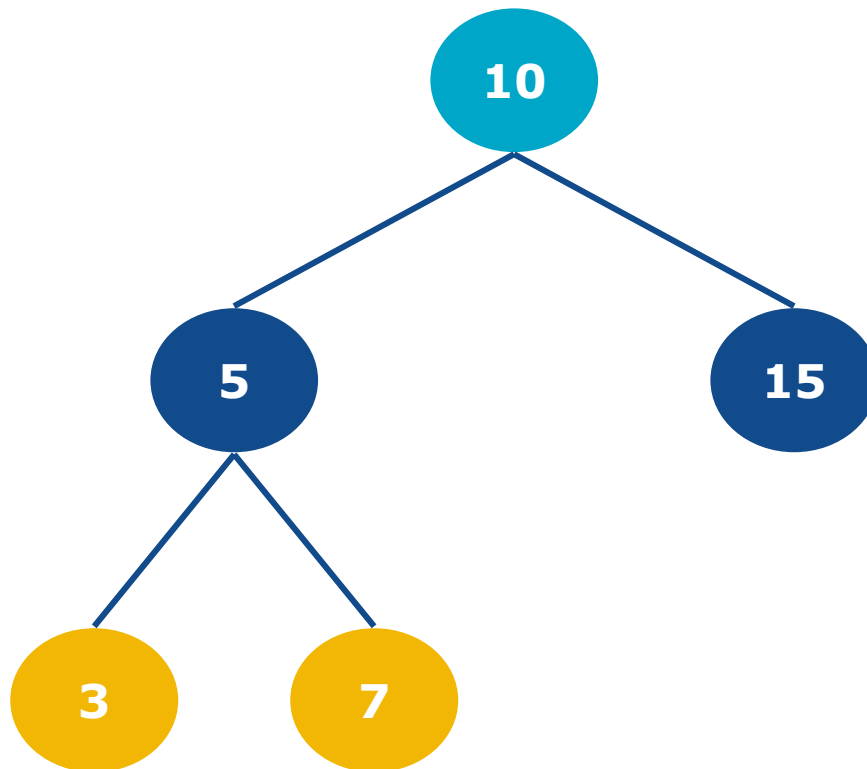
10

Pai

10 é pai de 5 e 15

Filho

5 é filho de 10



Folha

3, 7 e 15

Irmãos

5 e 15

Subárvore

5 com 3 e 7

Árvore binária

Cada nó possui no máximo dois filhos

Filho à esquerda

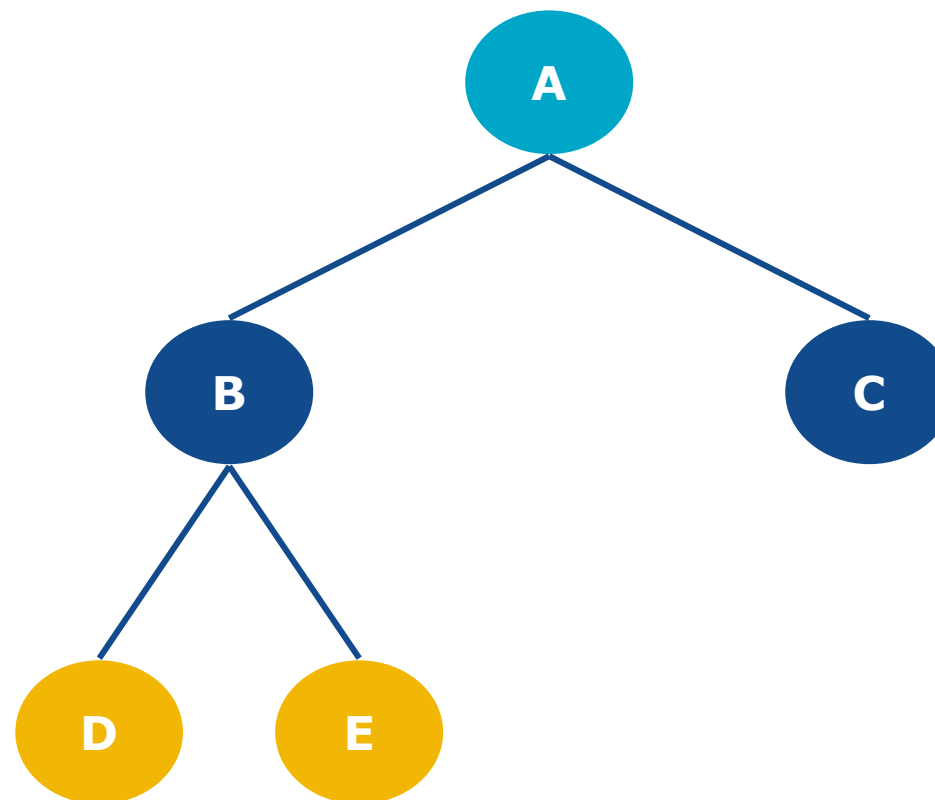
Pode existir ou não.

Filho à direita

Pode existir ou não.

Ordem dos valores

Uma árvore binária comum não exige ordenação.



0, 1 ou 2 filhos por nó

Árvore Binária de Busca

A posição de cada valor orienta a procura

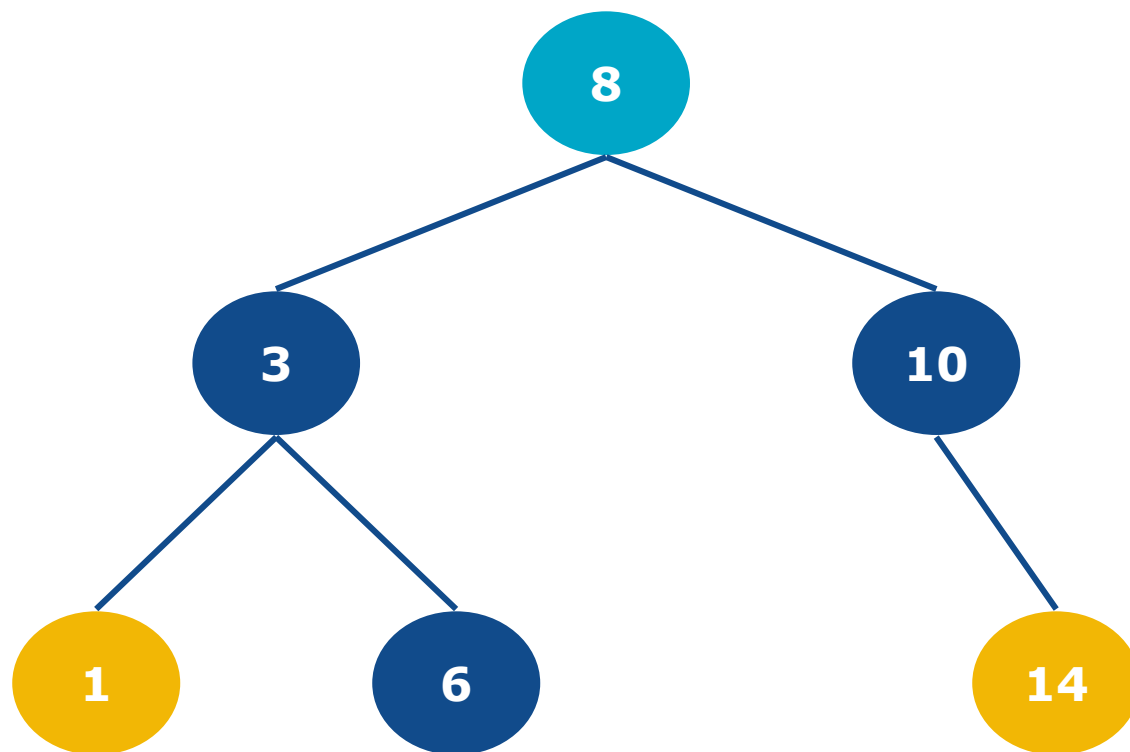
Menores

Valores menores que o nó seguem para a esquerda.

Maiores

Valores maiores que o nó seguem para a direita.

Convenção desta aula: sem valores duplicados.



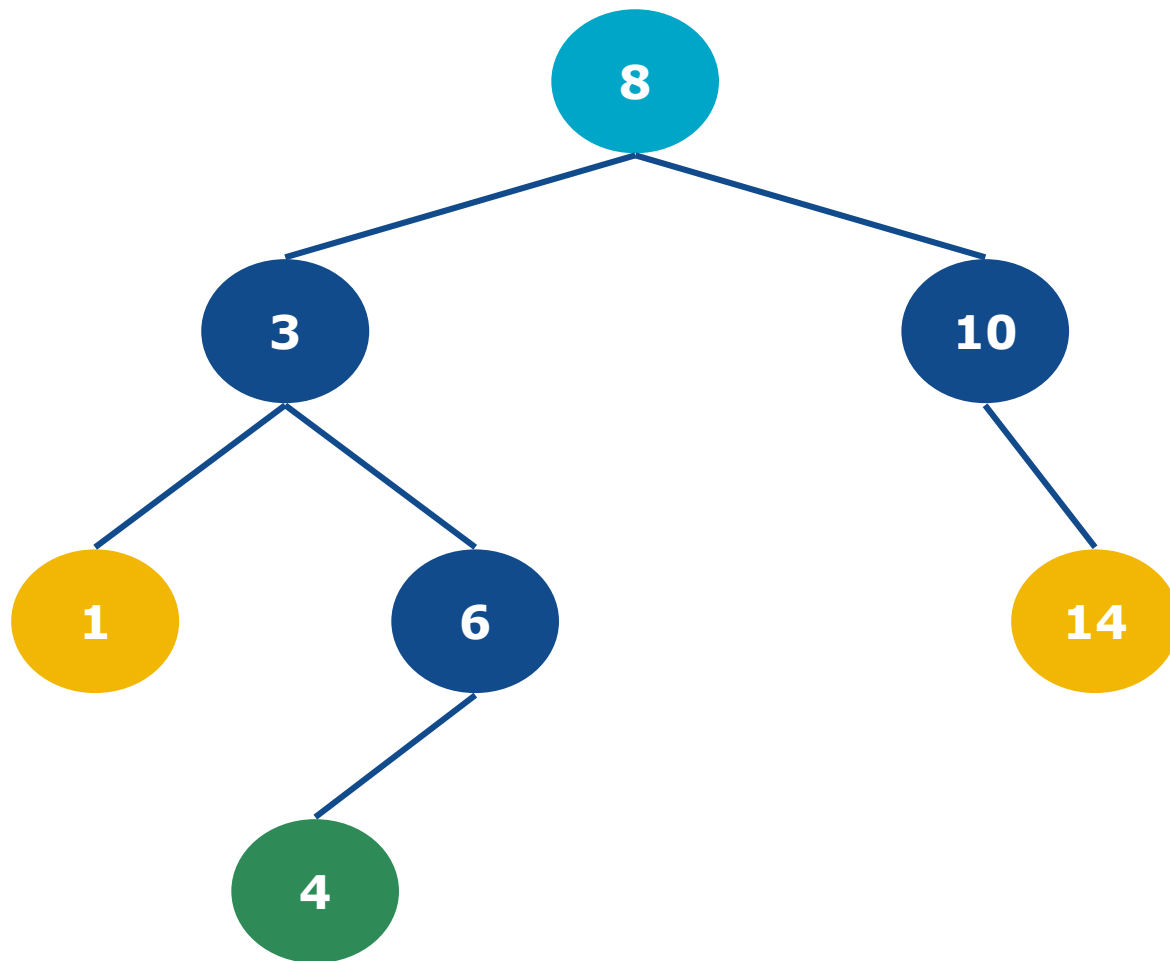
Inserção guiada

Sequência: 8, 3, 10, 1, 6, 14, 4

Para inserir 4

- $4 < 8$: siga à esquerda.
- $4 > 3$: siga à direita.
- $4 < 6$: siga à esquerda.
- Posição vazia: insira o novo nó.

A comparação se repete até encontrar uma posição vazia.



Busca básica em uma ABB

Cada comparação escolhe apenas uma subárvore

1

Compare

O valor é igual ao nó atual?

2

Decida

Menor: esquerda. Maior: direita.

3

Repita

Continue até encontrar o valor ou chegar a vazio.

Paradas

**Encontro
u**

ou

**chegou a
None**

Exemplo de busca

Procurando o valor 6

Comparação 1

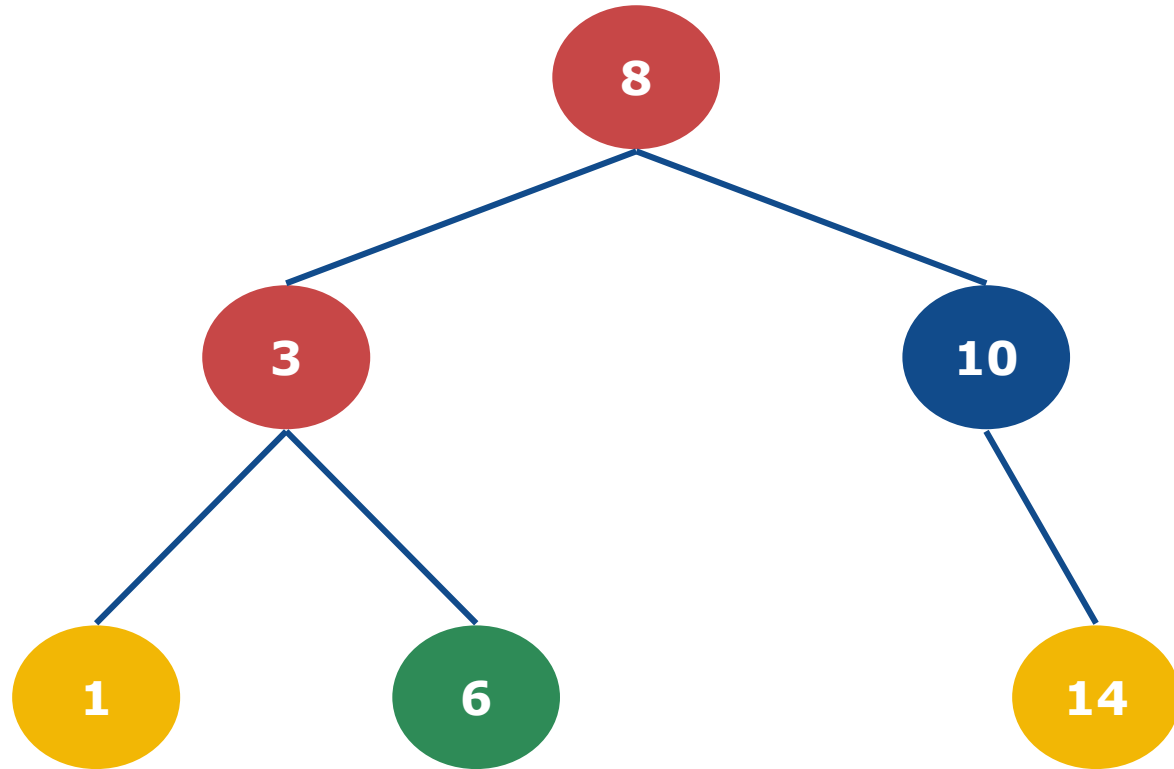
$6 < 8$: esquerda

Comparação 2

$6 > 3$: direita

Comparação 3

$6 = 6$: encontrado

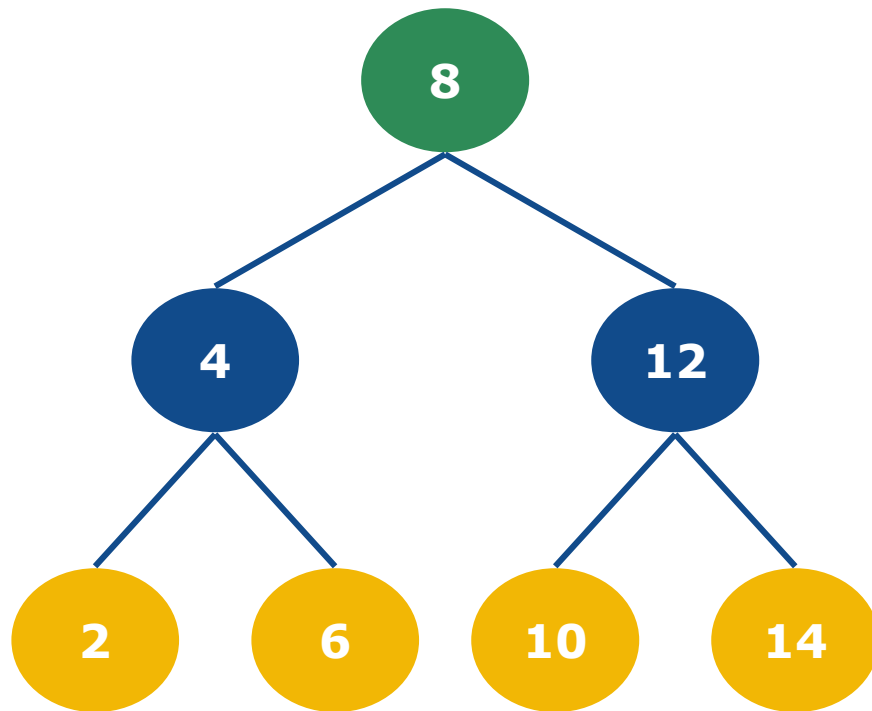


Caminho visitado: 8, 3, 6

Forma da árvore e custo da busca

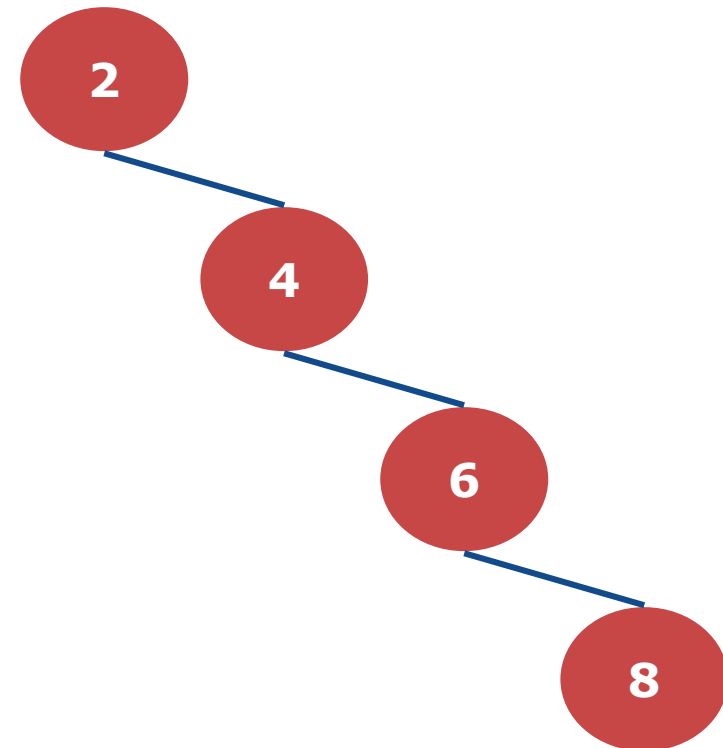
A ordem de inserção influencia o número de comparações

Mais equilibrada



Busca tende a $O(\log n)$

Degenerada



Pior caso: $O(n)$

Representação de um nó em Python

Cada nó guarda um valor e duas referências

```
class No:  
    def __init__(self, valor):  
        self.valor = valor  
        self.esquerda = None  
        self.direita = None
```

valor

Dado armazenado no nó

esquerda

Referência para valores menores

direita

Referência para valores maiores

Função de busca em Python

Versão iterativa

```
def buscar(raiz, alvo):  
    atual = raiz  
  
    while atual is not None:  
        if alvo == atual.valor:  
            return atual  
        if alvo < atual.valor:  
            atual = atual.esquerda  
        else:  
            atual = atual.direita  
  
    return None
```

Laço

Avança por um caminho da árvore.

Decisão

Compara o alvo com o nó atual.

Retorno

Entrega o nó ou None.

Teste rápido

Construa e explique o caminho de busca

Insira nesta ordem

50, 30, 70, 20, 40, 60, 80

Depois responda

- Qual é a raiz?
- Quais nós são folhas?
- Qual caminho a busca percorre para encontrar 60?
- Qual caminho ocorre ao buscar 65?

Tempo: 6 minutos • Duplas

Workshop prático

Design de uma experiência de busca em árvore

Desafio do grupo

Criar uma mecânica simples que ensine inserção ou busca em uma ABB por meio de decisões do jogador.

Grupos de 3 a 4 estudantes • 30 minutos

1

Situação base

Catálogo, mapa, inventário ou jogo

2

Regra da árvore

Menor à esquerda, maior à direita

3

Interação

Escolhas, feedback e condição de conclusão

Entregável do grupo

Mini GDD em uma página ou um slide

Problema

O que o jogador precisa localizar ou organizar?

Árvore

Quais valores formam a ABB inicial?

Core loop

Comparar, escolher um lado e avançar.

Feedback

Como o jogo indica decisão correta ou incorreta?

Vitória

Qual condição demonstra que a busca terminou?

Exemplo

Mostre uma busca completa passo a passo.

O modelo deve ensinar a regra da ABB, não apenas decorar a atividade.

Critérios de avaliação

Três dimensões para orientar o desenvolvimento

Rigor conceitual

A árvore respeita a ordenação da ABB. O caminho de busca e a condição de parada estão corretos.

Clareza da mecânica

O estudante entende o que comparar, qual decisão tomar e que feedback recebe.

Aplicação

A proposta conecta a estrutura de dados a uma situação compreensível e apresenta um exemplo completo.

Escala sugerida: excelente, satisfatório ou precisa de revisão

Pitches e feedback

Cada grupo dispõe de até 3 minutos

1

Qual problema o jogo resolve?

2

Como a ABB aparece na mecânica?

3

Demonstre uma busca.

4

Que aprendizado o feedback reforça?

Feedback da turma: uma força conceitual e uma sugestão de melhoria

Síntese e próximos passos

O que deve permanecer após a aula

- Árvores organizam dados por relações hierárquicas.
- Árvores binárias limitam cada nó a dois filhos.
- Uma ABB usa comparações para orientar inserção e busca.
- A forma da árvore altera o custo da operação.
- Na próxima aula: percursos em ordem, pré-ordem e pós-ordem.

Take Away!

**Em uma frase,
explique por que a
busca em uma ABB
pode visitar menos
nós do que uma busca
linear.**

Até a próxima aula!